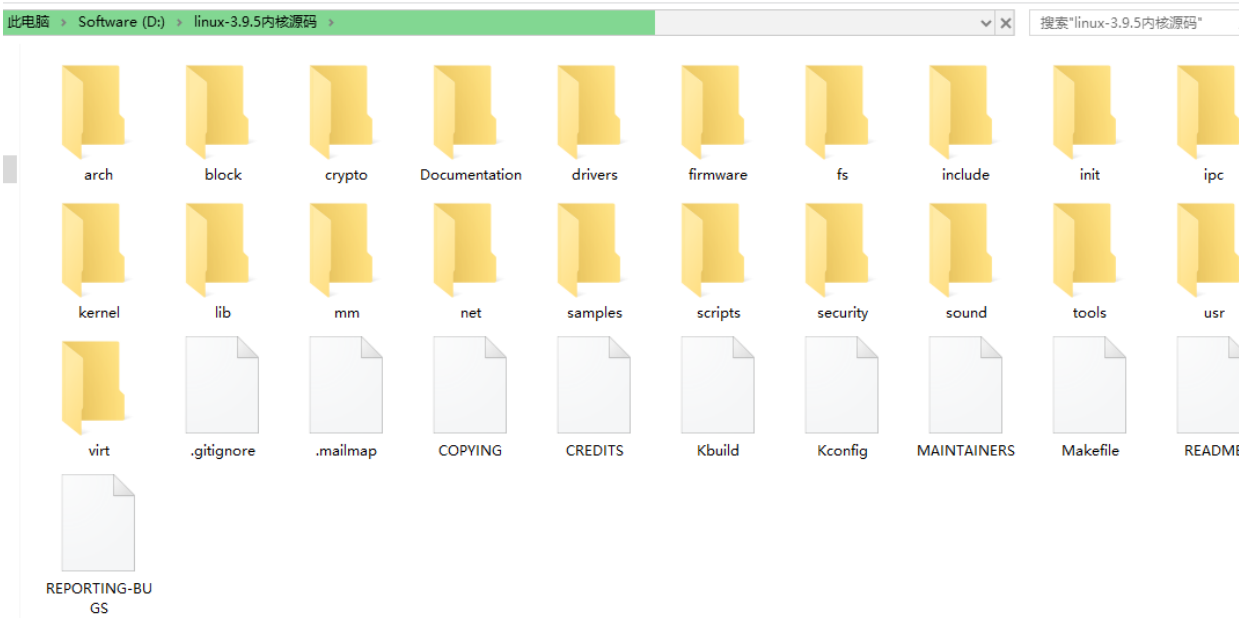
一、内核网络协议栈架构

Linux 内核层还提供网络防火墙的框架 netfilter，基于 netfilter 框架编写网络过滤程序是 Linux 环境下内核层网络处理的常用方法。

1、Linux 内核源代码结构

Linux 的内核源代码可以从 https://www.kernel.org/网站上下载，Linux-3.9.5 以上的版本(含)。 其代码目录结构如下：



❖ Documentation：这个目录下面没有内核的代码，有一套有用的内核文档。其中文档质量良莠不齐，有很多内核文档的质量很优秀并且相当完整，例如文件系统；但是有的则完全没有文档，例如进程调度。在这个目录里不时可以

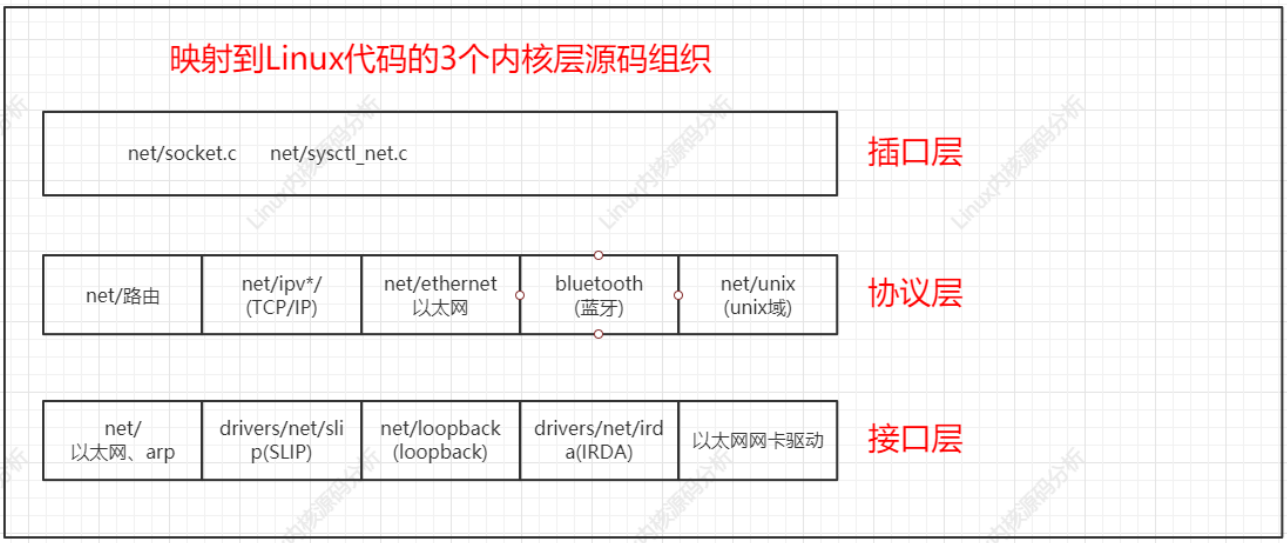
发现有用的东西。

- arch：此目录下的所有子目录的东西都是体系结构特有的代码。
- drivers：内核的驱动程序代码。此部分的代码占内核代码的大部分，包括显卡、网卡、PCI 等外围设备的驱动代码。
- fs：文件系统代码。包含 ext2、ext3、ext4 等本地文件系统，CD-ROM、isofs 等镜像系统，还有 NFS 等网络文件系统，以及 proc 等伪文件系统。
- include：此目录中包含了 Linux 内核中的大部分头（*.h）文件。
- init：内核初始化过程的代码。
- ipc：进程间通信代码。
- kernel：这部分是 Linux 内核中最重要的，包含了内核中平台无关的基本功能，主要包含进程创建、销毁和调度的代码。
- lib：此目录中主要包含内核中其他模块使用的通用函数和内核自解压的函数。
- mm：此目录中的代码实现了平台无关的内存管理代码。

- scripts：此目录下是内核配置时使用的脚本，当使用 make menuconfig 或者 make xconfig 命令时，会调用此部分代码。

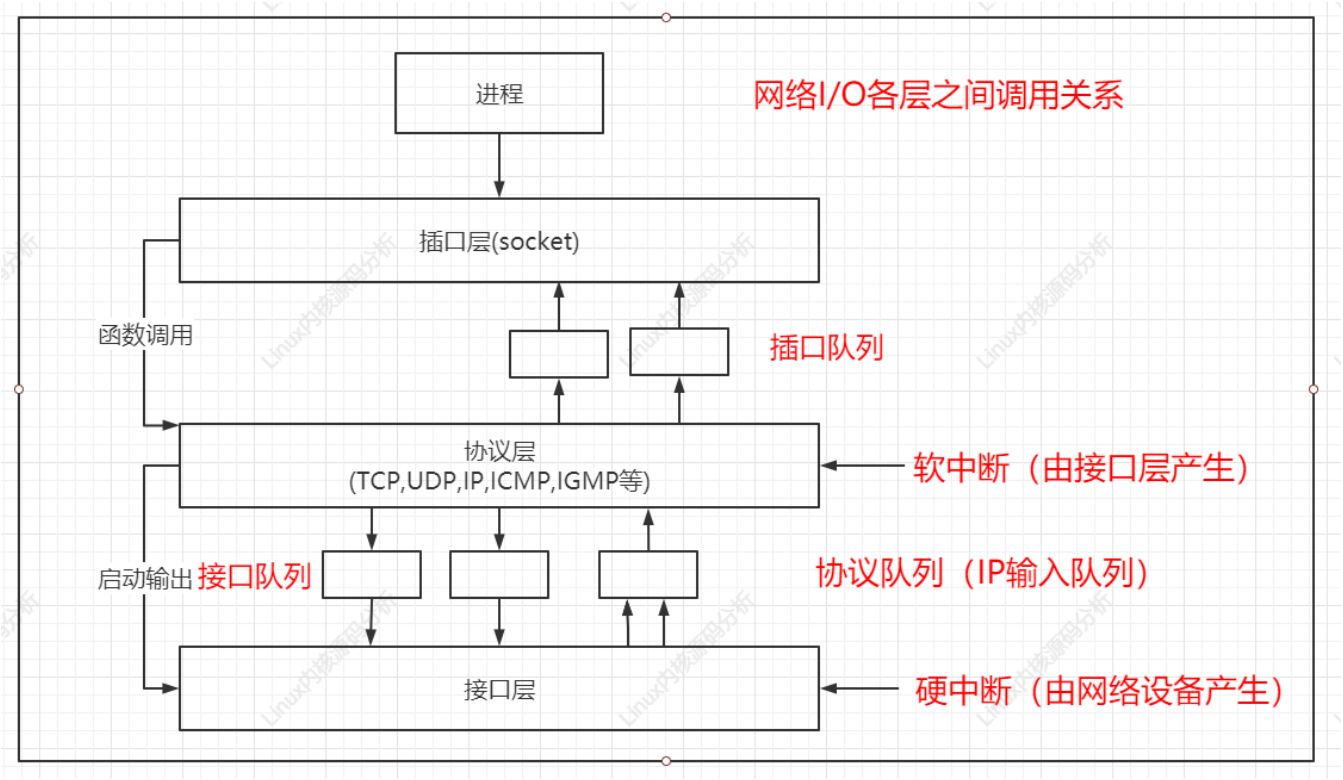
- net：此目录中包含 Linux 内核的网络协议栈的代码。在子目录 netfilter 下为 netfilter 的实现代码，netfilter 构建了一个框架，允许在不重新编译内核的情况下，编写可加载内核，在指定的地方插入回调函数，以用户自己的方式处理网络数据。子目录 ipv4 和 ipv6 为 TCP/IP 协议栈的 IPv4 和 IPv6 的实现，主要包含了 TCP、UDP、IP 协议的代码，还有 ARP 协议、ICMP 协议、IGMP 协议、netfilter 的 TCP/IP 实现等代码实现，以及如 proc、ioctl 等控制相关的代码。

组织代码另一个表现形式就是映射到 Linux 代码的 3 个内核层：



2、内核中网络剖析流程

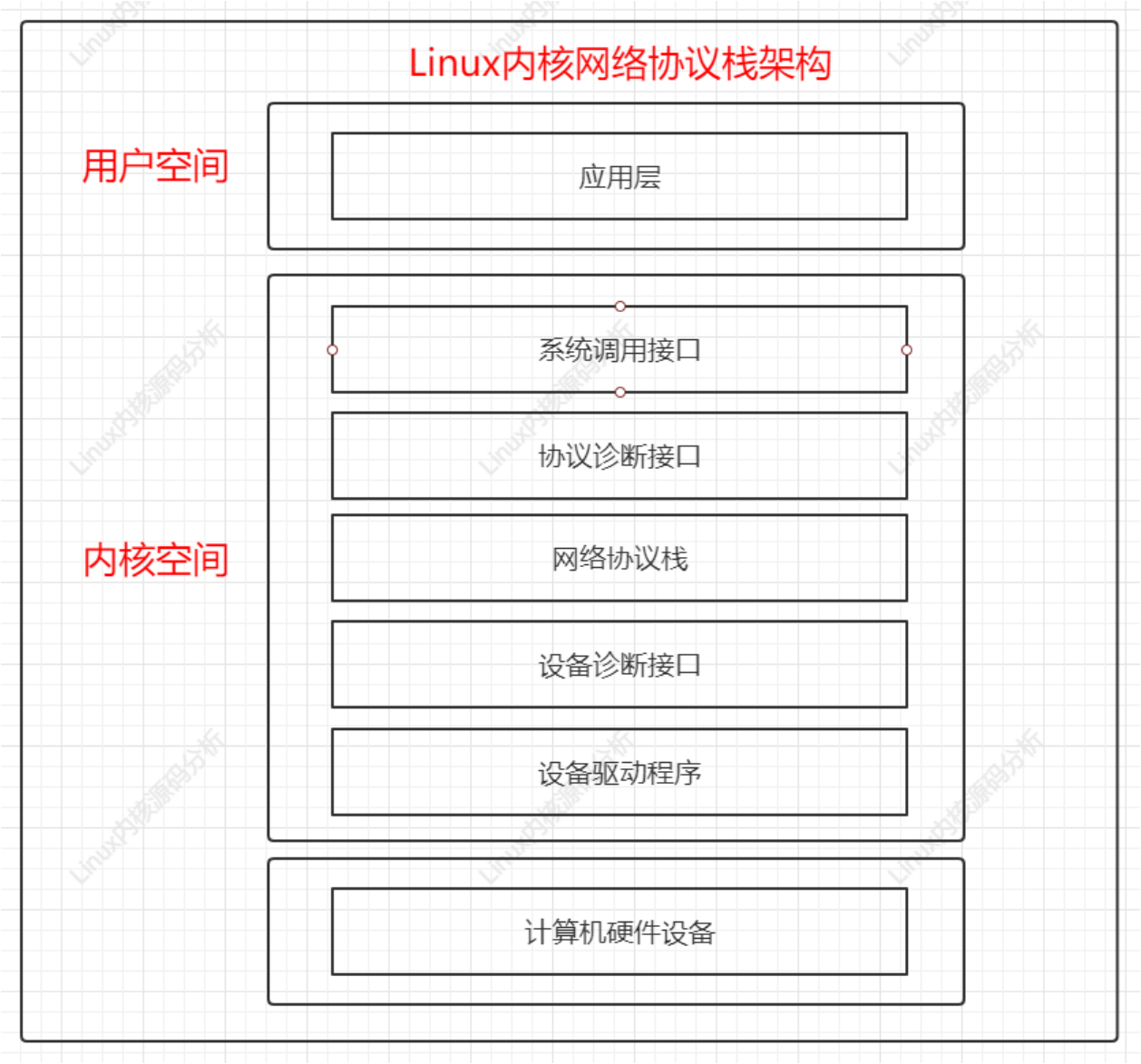
网络协议栈是由若干个层组成的，网络数据的流程主要是指在协议栈的各个层之间的传递。在前面章节中 TCP 网络编程的流程，一个 TCP 服务器的流程按照建立 socket()函数，绑定地址端口 bind()函数，侦听端口 listen()函数，接收连接 accept()函数，发送数据 send()函数，接收数据 recv()函数，关闭 socket()函数的顺序来进行。与此对应内核的处理过程也是按照此顺序进行的，网络数据在内核中的处理过程主要是在网卡和协议栈之间进行：从网卡接收数据，交给协议栈处理；协议栈将需要发送的数据通过网络发出去。如下图所示，总结了各层间在网络输入输出时的层间调用关系。由图中可以看出，数据的流向主要有两种。应用层输出数据时，数据按照自上而下的顺序，依次通过插口层、协议层和接口层；当有数据到达的时候，自下而上依次通过接口层、协议层和插口层的方式，在内核层传递。



应用层 Socket 的初始化、绑定（bind）和销毁是通过调用内核层的 socket()函数进行资源的申请和销毁的。发送数据的时候，将数据由插口层传递给协议层，协议层在 UDP 层添加 UDP 的首部、TCP 层添加 TCP 的首部、IP 层添加 IP 的首部，接口层的网卡则添加以太网相关的信息后，通过网卡的发送程序发送到网络上。

接收数据的过程是一个相反的过程，当有数据到来的时候，网卡的中断处理程序将数据从以太网网卡的 FIFO 对列中接收到内核，传递给协议层，协议层在 IP 层剥离 IP 的首部、UDP 层剥离 UDP 的首部、TCP 层剥离 TCP 的首部后传递给插口层，插口层查询 socket 的标识后，将数据送给用户层匹配的 socket。

如下图所示为 Linux 内核层的网络协议栈的架构视图。最上面是用户空间层，应用层的程序位于此处。最底部是物理设备，例如以太网网卡等，提供网络数据的连接、收发。中间是内核层，即网络协议栈子系统。流经网络栈内部的是 socket 缓冲区（由结构 sk_buffs 接连），它负责在源和汇点之间传递报文数据。



顶部（参见上图所示）是系统调用接口，它为用户

空间的应用程序提供了一种访问内核网络子系统的接口。位于其下面的是一个协议无关层，它提供了一种通用方法来使用底层传输层协议。然后是实际协议，在 Linux 中包括内嵌的协议 TCP、UDP，当然还有 IP。然后是另外一个网络设备协议无关层，提供了与各个设备驱动程序通信的通用接口，最下面是设备驱动程序本身。

二、结构 sk_buff 的原型

内核层和用户层在网络方面的差别很大，在内核的网络层中 sk_buff 结构占有重要的地位，几乎所有的处理均与此结构有关系。网络协议栈是一个层次架构的软件结构，层与层之间通过预定的接口传递报文。网络报文中包含了在协议各层使用到的各种信息。由于网络报文之间的大小不是固定的，因此采用合适的数据结构来存储这些网络报文就显得非常重要。

1．结构 sk_buff 的原型

sk_buff 数据结构的代码如下所示。

skbuff.h 3

include > linux > skbuff.h > sk_buff

```c

struct sk_buff_head {
    /* These two members must be first. */
    struct sk_buff  *next;
    struct sk_buff  *prev;

    __u32       qlen;
    spinlock_t  lock;
};

struct sk_buff;
```

```c
struct sk_buff {

    /* These two members must be first. */

    struct sk_buff      *next;

    struct sk_buff      *prev;



    ktime_t         tstamp;
```

```c
    struct sock     *sk;

    struct net_device   *dev;
```

```c
    /*
```

```
     * This is the control buffer. It is free to u
se for every
     * layer. Please put your private variables th
ere. If you
     * want to keep them across layers you have to
 do a skb_clone()
     * first. This is owned by whoever has the skb
 queued ATM.
     */
    char            cb[48] __aligned(8);
```

```
    unsigned long       _skb_refdst;
#ifdef CONFIG_XFRM
    struct  sec_path    *sp;
#endif
    unsigned int        len,
                data_len;
    __u16           mac_len,
                hdr_len;
```

```
union {
    __wsum      csum;
    struct {
        __u16   csum_start;
        __u16   csum_offset;
    };
};
__u32           priority;
kmemcheck_bitfield_begin(flags1);
__u8            local_df:1,
            cloned:1,
            ip_summed:2,
            nohdr:1,
            nfctinfo:3;
__u8            pkt_type:3,
            fclone:2,
            ipvs_property:1,
            peeked:1,
```

```
                nf_trace:1;

    kmemcheck_bitfield_end(flags1);

    __be16          protocol;
```

```
    void            (*destructor)(struct sk_buff *
skb);

#if defined(CONFIG_NF_CONNTRACK) || defined(CONFIG
_NF_CONNTRACK_MODULE)

    struct nf_conntrack *nfct;

#endif

#ifdef NET_SKBUFF_NF_DEFRAG_NEEDED

    struct sk_buff      *nfct_reasm;

#endif

#ifdef CONFIG_BRIDGE_NETFILTER

    struct nf_bridge_info   *nf_bridge;

#endif
```

```
    int         skb_iif;
```

```
    __u32           rxhash;
```

```
    __u16           vlan_tci;
```

```
#ifdef CONFIG_NET_SCHED

    __u16           tc_index;   /* traffic control
 index */

#ifdef CONFIG_NET_CLS_ACT

    __u16           tc_verd;    /* traffic control
 verdict */

#endif

#endif
```

```
    __u16           queue_mapping;

    kmemcheck_bitfield_begin(flags2);

#ifdef CONFIG_IPV6_NDISC_NODETYPE

    __u8            ndisc_nodetype:2;

#endif

    __u8            pfmemalloc:1;
```

```
    __u8            ooo_okay:1;
    __u8            l4_rxhash:1;
    __u8            wifi_acked_valid:1;
    __u8            wifi_acked:1;
    __u8            no_fcs:1;
    __u8            head_frag:1;
    /* Encapsulation protocol and NIC drivers shou
ld use
     * this flag to indicate to each other if the
skb contains
     * encapsulated packet or not and maybe use th
e inner packet
     * headers if needed
     */
    __u8            encapsulation:1;
    /* 7/9 bit hole (depending on ndisc_nodetype p
resence) */
    kmemcheck_bitfield_end(flags2);
```

```
#ifdef CONFIG_NET_DMA
    dma_cookie_t        dma_cookie;
#endif
#ifdef CONFIG_NETWORK_SECMARK
    __u32           secmark;
#endif
    union {
        __u32       mark;
        __u32       dropcount;
        __u32       reserved_tailroom;
    };
```

```
    sk_buff_data_t      inner_transport_header;
    sk_buff_data_t      inner_network_header;
    sk_buff_data_t      transport_header;
    sk_buff_data_t      network_header;
    sk_buff_data_t      mac_header;
```

```
    /* These elements must be at the end, see allo
c_skb() for details.  */
    sk_buff_data_t      tail;
    sk_buff_data_t      end;
    unsigned char       *head,
                *data;
    unsigned int        truesize;
    atomic_t        users;
};
```

sk_buff 结构主要成员的含义如下所述：

- next：sk_buff 链表中的下一个缓冲区。
- prev：sk_buff 链表中的前一个缓冲区。以上两个变量将 sk_buff 链接到一个双向链表中。
- sk：本网络报文所属的 sock 结构，此值仅在本机发出的报文中有效，从网络收到的报文此值为空。
- tstamp：报文收到的时间戳。
- dev：收到此报文的网络设备。
- transport_header：传输层头部。

- network_header：网络层头部。
- mac_header：链接层头部。
- cb：用于控制缓冲区。每个层都可以使用此指针，将私有的数据放置于此。
- len：有效数据长度。
- data_len：数据长度。
- mac_len：连接层头部长度，对于以太网，指 MAC 地址所用的长度，为 6。
- hdr_len：skb 的可写头部长度。
- csum：校验和（包含开始和偏移）。
- csum_start：当开始计算校验和时从 skb->head 的偏移。
- csum_offset：从 csum_start 开始的偏移。
- local_df：允许本地分片。
- pkt_type：包的类别。
- priority：包队列的优先级。
- truesize：报文缓冲区的大小。
- head：报文缓冲区的头。

- data：数据的头指针。
- tail：数据的尾指针。
- end：报文缓冲区的尾部

网络报文存储空间是在应用层发送网络数据或者网络设备收到网络数据时动态分配的，分配成功之后，将接收或者发送的网络数据填充到这个存储空间中去。将网络数据填充到存储空间时，在存储空间的头部预留了一定数量的空隙，然后从此偏移量开始将网络报文复制到存储空间中。

进程的用户栈和内核栈

进程是程序的一次执行过程。用剧本和演出来类比，程序相当于剧本，而进程则相当于剧本的一次演出，舞台、灯光则相当于进程的运行环境。

进程的堆栈

每个进程都有自己的堆栈，内核在创建一个新的进程时，在创建进程控制块 task_struct 的同时，也为进程创建自己堆栈。一个进程 有 2 个堆栈，用户堆栈和系统堆栈；用户堆栈的空间指向用户地址空间，内核堆栈的空间指向内核地址空间。当进程在用户态运行时，CPU 堆栈指针寄存器指向的用户堆栈地址，使用用户堆栈，当进程运行在内核态时，CPU 堆栈指针寄存器指向的是内核栈空间地址，使用的是内核

栈；

进程用户栈和内核栈之间的切换

当进程由于中断或系统调用从用户态转换到内核态时，进程所使用的栈也要从用户栈切换到内核栈。系统调用实质就是通过指令产生中断，称为软中断。进程因为中断（软中断或硬件产生中断），使得 CPU 切换到特权工作模式，此时进程陷入内核态，进程进入内核态后，首先把用户态的堆栈地址保存在内核堆栈中，然后设置堆栈指针寄存器的地址为内核栈地址，这样就完成了用户栈向内核栈的切换。

当进程从内核态切换到用户态时，最后把保存在内核栈中的用户栈地址恢复到 CPU 栈指针寄存器即可，这样就完成了内核栈向用户栈的切换。

这里要理解一下内核堆栈。前面我们讲到，进程从用户态进入内核态时，需要在内核栈中保存用户栈的地址。那么进入内核态时，从哪里获得内核栈的栈指针呢？

要解决这个问题，先要理解从用户态刚切换到内核态以后，进程的内核栈总是空的。这点很好理解，当进程在用户空间运行时，使用的是用户 栈；当进程在内核态运行时，内核栈中保存进程在内核态运行的相关信息，但是当进程完成了内核态的运行，重新回到用户态时，此时内核栈中保存的信息全部恢 复，也就是说，进程在内核态中的代码执行完成回到用户态时，内核栈是空的。

理解了从用户态刚切换到内核态以后，进程的内核栈总是空的，那刚才这个问题就很好理解了，因为内核栈是空的，那当进程从用户态切换到内核态后，把内核栈的栈顶地址设置给 CPU 的栈指针寄存器就可以了。

X86 Linux 内核栈定义如下（可能现在的版本有所改变，但不妨碍我们对内核栈的理解）：

在／include/linux/sched.h 中定义了如下一个联合结构：

```
union task_union {
struct task_struct task;
unsigned long stack[2408];
};
```

从这个结构可以看出，内核栈占 8kb 的内存区。实际上，进程的 task_struct 结构所占的内存是由内核动态分配的，更确切地说，内核根本不给 task_struct 分配内存，而仅仅给内核栈分配 8K 的内存，并把其中的一部分给 task_struct 使用。这样内核栈的起始地址就是 union task_union 变量的地址 +8K 字节的长度。例如：我们动态分配一个 union task_union 类型的变量如下：

```
unsigned char *gtaskkernelstack
gtaskkernelstack = kmalloc(sizeof(union task_union));
```

那么该进程每次进入内核态时，内核栈的起始地址均为：（unsigned char *）gtaskkernelstack + 8096

进程上下文

进程切换现场称为进程上下文(context)，包含了一个进程所具有的全部信息，一般包括：进程控制块（Process Control Block，PCB）、有关程序段和相应的数据集。

进程控制块 PCB（任务控制块）

进程控制块是进程在内存中的静态存在方式，Linux 内核中用 task_struct 表示一个进程（相当于进程的人事档案）。进程的静 态描述必须保证一个进程在获得 CPU 并重新进入运行态时，能够精确的接着上次运行的位置继续进行，相关的程序段，数据以及 CPU 现场信息必须保存。处理机 现场信息主要包括处理机内部寄存器和堆栈等基本数据。

进程控制块一般可以分为进程描述信息、进程控制信息，进程相关的资源信息和 CPU 现场保护机构。

进程的切换

当一个进程的时间片到时，进程需要让出 CPU 给其他进程运行，内核需要进行进程切换。

Linux 的进程切换是通过调用函数进程切换函数 schedule 来实现的。进程切换主要分为 2 个步骤：

1. 调用 switch_mm()函数进行进程页表的切换；
2. 调用 switch_to() 函数进行 CPU 寄存器切换；

__switch_to 定义在/arch/arm/kernel 目录下的 entry-armv.S 文件中，源码如下：

Switch_to 的处理流程如下：

1. 保存本进程的 CPU 寄存器（PC、R0 ~ R13）到本进程的栈中；
2. 保存 SP(本进程的栈基地址)到 task->thread.save 中；
3. 从新进程的 task->thread.save 恢复 SP 为新进程的栈基地址；
4. 从新进程的栈中恢复新进程的 CPU 相关寄存器值，
5. 新进程开始运行，完成任务切换。

这里读者可能会问，在进行任务切换的时候，到底是在运行进程 1 还是运行进程 2 呢？进程切换的时候，已经进行页表切换，那页表切换之后，切换进程使用的是进程 1 还是进程 2 的页表呢？

要回答这个问题，首先我们要明白由谁来完成进程切换？

通过对操作系统的理解，毫无疑问，进程切换是由内核来完成的，也就是说，在进行进程切换时，CPU 运行在内核模式，使用的是内核空间的内核代码，它既不属于进程也不属于进程 2，当进程的时间片到时，内核提供服务来完成进程的切

换。既不使用进程 1 的页表，也不使用进程 2 的页表，使用的内核映射页表。这样我们就很好理解上面的问题了。

三、协议栈中软中断架构

a. Linux 内核中软中断的机制 473

b. 网络收发处理软中断实现机制 475(画协议栈中的软中断架构图)

c. 网卡接收数据流程 476 画图

d1. 协议栈处理数据流程 16.12 画图

d2. 协议栈处理数据流程 16.13 画图

四、Iptables 和 Netfilter

a. 基于 Netfileter 框架在 Linux 的内核层挂接自己的网络数据处理函数，对内核层网络数据进行过滤如何使用 Iptables 控制 netfilter。